
Marco de niveles

La aplicación que genera aplicaciones

Documento fundacional del proyecto SLCP

Documento **SLCP-DOC-000** – Versión **1.0**

6 de agosto de 2026

SLCP no es una aplicación: es una aplicación que produce aplicaciones. Esa condición de metasistema atraviesa todas las decisiones del proyecto y es la fuente más probable de error de diseño, porque cada requisito puede referirse a tres cosas distintas y la ambigüedad no siempre es visible al leerlo. Este documento fija los tres niveles, establece la regla de redacción que obliga a declararlos, y registra la auditoría de los requisitos ya emitidos en SLCP-DOC-001 que resultaban ambiguos, con su resolución.

Índice

1	El objeto del proyecto	2
2	Los tres niveles	2
3	Consecuencia arquitectónica: dos metamodelos	2
4	Regla de redacción de requisitos	3
5	Auditoría de SLCP-DOC-001	3
5.1	Conflación 1: idioma de los identificadores	3
5.2	Conflación 2: alcance de las restricciones de licencia	4
5.3	Ambigüedades menores	4
5.4	Requisito nuevo derivado de la auditoría	4
6	Autoaplicación	4
7	Errores típicos que este marco previene	5

1 El objeto del proyecto

SLCP es un generador de aplicaciones. Recibe una especificación de requisitos y produce, sobre ella, los artefactos del ciclo de vida de una aplicación distinta de sí misma, incluido su código fuente y el de su lógica de negocio. Todo cuanto se especifique en este proyecto debe leerse a la luz de ese propósito: la plataforma no resuelve un dominio, resuelve el problema de construir soluciones para dominios arbitrarios.

De aquí se deriva la propiedad que gobierna la arquitectura: **el dominio de la aplicación generada es dato para SLCP, no código**. Incorporar un dominio nuevo — matrícula académica, inventario, historia clínica — no puede exigir ninguna modificación del núcleo. Si en algún momento lo exige, el diseño ha fallado, con independencia de que el resultado funcione.

2 Los tres niveles

Tabla 1: Niveles de abstracción del proyecto

Nivel	Qué es	Qué contiene	Quién lo construye
N1	La plataforma SLCP	Código del núcleo, motor de plantillas, almacén de trazabilidad, interfaz de usuario	El equipo del proyecto, a mano
N2	El proyecto gestionado dentro de SLCP	Requisitos, modelo de dominio, reglas de negocio, oráculos, enlaces de trazabilidad de la aplicación por construir	La persona usuaria de SLCP, asistida por SLCP
N3	La aplicación generada	Código fuente, esquema, migraciones, contratos, pruebas y documentación de la aplicación destino	SLCP, a partir de N2

La confusión más frecuente se produce entre N1 y N3, porque ambos son código y ambos se compilan, se prueban y se despliegan. Pero obedecen a reglas distintas: N1 está sujeto a las restricciones de construcción que fija el proyecto, mientras que N3 está sujeto a las decisiones de quien encarga la aplicación generada. La segunda confusión, más sutil, se produce al olvidar que N2 no es documentación sino *datos de N1*: los requisitos, el modelo y las reglas son instancias almacenadas por la plataforma, no archivos que la acompañan.

3 Consecuencia arquitectónica: dos metamodelos

SLCP manipula simultáneamente dos metamodelos que no deben mezclarse.

El **metamodelo del ciclo de vida** es fijo y lo define SLCP-DOC-001: *Requirement*, *BusinessRule*, *TestCase*, *TraceLink* y demás entidades del vocabulario canónico. Es el mismo para todos los proyectos y no cambia cuando cambia el dominio.

El **metamodelo del dominio** es variable y pertenece a cada proyecto de nivel N2: Estudiante, Matrícula, Paciente. Para SLCP no son tipos: son instancias de una entidad del primer metamodelo. La plataforma nunca conoce el concepto de estudiante; conoce el concepto de entidad de dominio, de la cual estudiante es un dato.

Esta separación tiene una consecuencia verificable, y por eso se convierte en requisito: si para dar soporte a un dominio nuevo hubiera que recompilar el núcleo, la separación se habría roto. La conformidad con Int [1] exige además que la descripción de arquitectura distinga de forma explícita las vistas de cada nivel, porque las partes interesadas de N1 y las de N3 son personas distintas con preocupaciones distintas.

4 Regla de redacción de requisitos

Todo requisito del proyecto debe declarar el nivel al que aplica mediante la marca [N1], [N2], [N3] o la combinación que corresponda. Un requisito sin marca de nivel se considera mal formado y no puede alcanzar línea base, por la misma razón por la que no puede hacerlo un requisito sin criterio de aceptación.

La marca no es burocracia: la mayoría de los errores detectados en la auditoría de la Sección 5 habrían sido visibles en el momento de redactar si la marca hubiera sido obligatoria desde el inicio.

5 Auditoría de SLCP-DOC-001

La revisión de los requisitos ya emitidos localizó dos conflaciones reales y varias ambigüedades de menor alcance. Se registran aquí con su resolución; las correcciones están aplicadas en SLCP-DOC-001 v1.2.

5.1 Conflación 1: idioma de los identificadores

NAM-01 exigía que todo identificador técnico se exprese en inglés. Leído sin marca de nivel, el requisito obliga a traducir al inglés el vocabulario de dominio de la aplicación generada, de modo que una entidad *Matrícula* se convertiría en *Enrollment* en el código de N3. Eso no es lo que se pretendía y sería además perjudicial: rompe la correspondencia entre el lenguaje de la parte interesada y el código que la representa, que es una de las pocas propiedades que hacen mantenible un sistema de gestión.

Resolución. NAM-01 aplica a [N1] y, en [N3], únicamente a los identificadores estructurales que SLCP introduce por su cuenta. Los identificadores derivados del dominio conservan el idioma del glosario del proyecto N2 y se someten solo a las convenciones de forma de NAM-03 en adelante. Se añade el requisito NAM-07, que obliga a mantener un glosario de dominio con un término canónico por concepto, del cual se derivan de forma determinista todos los identificadores de N3.

5.2 Conflación 2: alcance de las restricciones de licencia

CON-01 exigía licencia aprobada por la OSI para todo componente incorporado. Leído sin marca de nivel, impondría esa licencia también a la aplicación generada y a sus dependencias, lo que la plataforma no tiene autoridad para decidir: esa elección corresponde a quien encarga la aplicación de N3.

Resolución. CON-01 a CON-05 aplican a [N1]. Para [N3] se añade CON-06, que obliga a la plataforma a permitir declarar la licencia del proyecto generado, a propagarla a sus artefactos y a emitir su inventario de componentes, sin imponer restricción alguna sobre cuál sea.

5.3 Ambigüedades menores

Tabla 2: Requisitos cuyo nivel se precisa

Requisito	Ambigüedad	Nivel asignado
TRC-10	No indicaba de qué repositorio se captura el vínculo entre requisito y código	[N2]: el repositorio del proyecto gestionado, no el de la plataforma
TRC-15	Las líneas base podían entenderse como las de la propia plataforma	[N2]: líneas base del proyecto gestionado
TRC-24	El almacén de solo anexo podía leerse como propiedad del proyecto generado	[N1]: es el almacén de la plataforma
TGT-06	Las medidas de calidad podían atribuirse al núcleo	[N3]: aplican al código generado; el núcleo se rige por sus propias pruebas
TGT-10	Las capas de verificación mezclaban ambos códigos	[N3], ejecutadas por [N1]
INT-09	La ida y vuelta podía entenderse sobre artefactos de N3	[N2]: el paquete que se exporta es el proyecto gestionado

5.4 Requisito nuevo derivado de la auditoría

TGT-14 [N1]. El núcleo de la plataforma no debe contener conocimiento de ningún dominio de aplicación. Es la extensión natural de TGT-05, que ya prohibía el conocimiento de lenguajes, gestores de bases de datos y frameworks destino. Criterio de verificación: dar soporte a un dominio nuevo se realiza cargando un proyecto N2, sin recompilar ni modificar el núcleo, y una búsqueda léxica del código de N1 no encuentra ningún término de dominio de los casos de prueba.

6 Autoaplicación

SLCP puede gestionarse a sí misma como proyecto de nivel N2, es decir, tratar su propia especificación como entrada de su propio motor. Es la prueba más exigente que existe del producto: una plataforma que no puede gestionar su propio desarrollo difícilmente podrá gestionar el de otros.

Conviene, sin embargo, no intentarlo demasiado pronto. Mientras el metamodelo siga cambiando, la autoaplicación obliga a migrar los datos del propio proyecto en cada cambio y multiplica el coste de cada iteración. La recomendación es diferirla hasta que el metamodelo se estabilice, y adoptarla entonces como criterio de aceptación de la versión mayor correspondiente.

7 Errores típicos que este marco previene

1. Codificar en el núcleo una entidad del dominio del primer caso de prueba, con lo que la plataforma deja de ser un generador y pasa a ser una aplicación de ese dominio con generación incorporada.
2. Aplicar a la aplicación generada restricciones que solo correspondían a la plataforma, imponiendo decisiones que no son del proyecto.
3. Confundir la trazabilidad que la plataforma *ofrece* con la trazabilidad de su propio desarrollo, y construir una sola cuando hacen falta dos.
4. Diseñar la interfaz de usuario en torno al dominio del primer cliente, en lugar de en torno al metamodelo del ciclo de vida.
5. Medir la calidad del código generado con los criterios del núcleo, o al revés.

Referencias

- [1] *ISO/IEC/IEEE 42010:2022 Software, systems and enterprise — Architecture description*. International Organization for Standardization, Geneva, Switzerland, 2022. <https://www.iso.org/standard/74393.html>.